

SKILL WEEK:-1

Session 1 :-

Task1:-

To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Make file.

Source code:

```
#include <stdio.h>

int main() {
    printf("Hello, OSSP Skill Week 1!\n");
    printf("GCC compilation is working successfully.\n");

    return 0;
}
```

OUTPUT:-

```
anuan@Anushka:~$ nano hello.c
anuan@Anushka:~$ gcc hello.c -o hello
anuan@Anushka:~$ ./hello
Hello, OSSP Skill Week 1!
GCC compilation is working successfully.
anuan@Anushka:~$
```

Git version:

```
anuan@Anushka:~$ git --version
git version 2.53.0
anuan@Anushka:~$
```

Observation:

I learned how to work with the Linux environment and verify system tools such as GCC and Git. I learned how to compile and execute a C program using GCC and understood the purpose of a Makefile for automating the compilation process. I also learned how Linux development projects can be organized and built using standard compiler and build commands.

Task2:

To Analyse process abstraction, execute fork(), understand exec() family, analyse parent-child relationships, inspect process tree, practice system call tracing.

Source code:

```

GNU nano 7.2 skill_week1_task2.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("Executing ls using exec()\n");
        execlp("ls", "ls", "-l", NULL);

        perror("exec failed");
        exit(1);
    } else {
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
        printf("Parent waiting for child...\n");

        wait(NULL);

        printf("Child process completed.\n");
    }

    return 0;
}

```

Output:-

```

anuran@anusha:~$ gcc process.c -o process
anuran@anusha:~$ ./process
Parent Process Started
Parent PID: 562
Child Process
Parent PID: 562
Child PID: 563
Parent waiting for child...
Child Process
Parent PID: 562
Child PID: 563
Executing ls using execlp()
total 28
-rwxr-xr-x 1 anuran anuran 169 Aug 7 09:26 file1.txt
-rwxr-xr-x 11 anuran anuran 4896 Aug 7 09:27 file2.txt
-rwxr-xr-x 11 anuran anuran 4896 Aug 7 09:29 Practical
-rwxr-xr-x 1 anuran anuran 4896 Aug 7 09:29 Skill
-rwxr-xr-x 1 anuran anuran 16408 Aug 8 03:24 command_executor
-rwxr-xr-x 1 anuran anuran 16216 Aug 7 09:27 file_copy
-rwxr-xr-x 1 anuran anuran 1113 Aug 7 09:27 file_copy.c
-rwxr-xr-x 1 anuran anuran 16088 Aug 8 03:25 file_copy.c.save
-rwxr-xr-x 1 anuran anuran 516 Aug 8 03:01 fork_exec.c
-rwxr-xr-x 1 anuran anuran 517 Aug 8 03:01 fork_exec.c.save
-rwxr-xr-x 1 anuran anuran 1209 Aug 8 03:05 fork_exec.r
-rwxr-xr-x 1 anuran anuran 16088 Aug 8 03:28 hardware_os
-rwxr-xr-x 1 anuran anuran 728 Aug 8 03:29 hardware_os.c
-rwxr-xr-x 1 anuran anuran 422 Aug 8 03:47 hello
-rwxr-xr-x 1 anuran anuran 1883 Aug 7 09:37 keyboard_input.c
-rwxr-xr-x 1 anuran anuran 16532 Aug 11 04:41 lex_parser
-rwxr-xr-x 1 anuran anuran 4239 Aug 11 04:43 lex_parser.c
-rwxr-xr-x 1 anuran anuran 16224 Aug 17 04:45 main_loop
-rwxr-xr-x 1 anuran anuran 559 Aug 17 04:04 main_loop.c
-rwxr-xr-x 1 anuran anuran 8844 Aug 11 04:34 parser.c
-rwxr-xr-x 1 anuran anuran 8844 Aug 11 04:37 parser_lexer.c
-rwxr-xr-x 1 anuran anuran 853 Aug 11 04:42 parser_lexer.c.save
-rwxr-xr-x 1 anuran anuran 16204 Aug 17 04:38 process
-rwxr-xr-x 1 anuran anuran 81 Aug 17 04:38 process.c
-rwxr-xr-x 1 anuran anuran 797 Aug 4 08:35 process.c.save
-rwxr-xr-x 1 anuran anuran 16088 Aug 8 03:22 process_states
-rwxr-xr-x 1 anuran anuran 1376 Aug 8 02:52 process_states.c
-rwxr-xr-x 1 anuran anuran 676 Aug 8 02:52 process.c
-rwxr-xr-x 1 anuran anuran 9 Aug 7 04:52 pwd
-rwxr-xr-x 1 anuran anuran 16226 Aug 8 03:04 state_monitor
-rwxr-xr-x 1 anuran anuran 788 Aug 8 03:04 state_monitor.c
Child Process Completed
anuran@anusha:~$

```

Observation:

I learned how Linux creates and manages processes using fork(). I understood that exec() can replace the child process with another program, while wait() allows the parent process to wait until the child process finishes.

Session2

Task1:

To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control

Flow Diagram, Test Interactive Loop Source code:

```
GNU nano 7.2 session2_task1.c
#include <stdio.h>
#include <string.h>

int main() {
    char command[100];

    printf("OSSP Interactive Shell\n");
    while (1) {
        printf("myshell> ");
        if (fgets(command, sizeof(command), stdin) == NULL) {
            break;
        }

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0) {
            printf("Exiting shell...\n");
            break;
        }

        if (strlen(command) == 0) {
            continue;
        }

        printf("You entered: %s\n", command);
    }

    return 0;
}
```

Output:

```
anuan@Anushka:~$ nano main_loop.c
anuan@Anushka:~$ gcc main_loop.c -o main_loop
anuan@Anushka:~$ ./main_loop
OSSP Interactive Shell
myshell> hello
You entered: hello
myshell> exit
Exiting shell...
anuan@Anushka:~$
```

Observation:

I learned how to create an interactive main loop in C that repeatedly displays a prompt and reads user input. I also learned how to handle exit conditions, manage input strings, and control program flow using loops and conditional statements.

Task2:

To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction.

Source code:

```
GNU nano 7.2
#include <stdio.h>

int main() {
    char buffer[100];
    int ch;
    int index = 0;

    printf("Type a command and press Enter:\n");
    printf("input> ");

    while ((ch = getchar()) != '\n' && ch != EOF) {
        if (ch == 127 || ch == 8) {
            if (index > 0) {
                index--;
            }
        } else if (index < 99) {
            buffer[index++] = ch;
        }
    }

    buffer[index] = '\0';

    printf("\nCommand entered: %s\n", buffer);
    printf("Characters stored: %d\n", index);

    return 0;
}
```

Output:

```
anuan@Anushka:~$ nano keyboard_input.c
anuan@Anushka:~$ gcc keyboard_input.c -o keyboard_input
anuan@Anushka:~$ ./keyboard_input
Keyboard Input Program
Type a command and press Enter.
Backspace = delete character, Ctrl+C = exit

>
Command received:
Empty command.
```

Observed:

learned how to capture keyboard input in C using `getchar()`. I understood how to manage an input buffer to store multiple characters, process the Enter key to finish input, and handle the Backspace key by removing the last character from the buffer. I also learned how user input can be processed and displayed as a complete command.